

Step 4 — A Router That Sends Each Message to the Right Agent Automatically — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (what a router should do when it genuinely doesn't know). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — A router is just a decision function in front of what you already built](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — A router is just a decision function in front of what you already built

Basic Version

You already have both real skills: chatting (Step 2) and extracting a ticket (Step 3). This step doesn't add a new skill — it adds a small decision-maker in front of them.

The **router** is just a function that looks at one incoming message and decides: "does this sound like a complaint that needs a ticket, or is this just normal conversation?" It can be as simple as checking for certain words, or as clever as one small, focused model call.

Once you have that decision, rename your two existing code paths as `concierge_agent(...)` and `triage_agent(...)`, and make `main.py`'s loop ask the router first, instead of deciding for itself.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The signature to build in `agents/router.py` :

```
def decide(message: str) -> Literal["concierge", "triage"]:
```

Two real ways to build it, both valid: 1. A **keyword check** — cheap, instant, no API call, but brittle on unclear wording. Make it case-insensitive: run `message.lower()` before checking, so `"Charged"` and `"charged"` route the same way. 2. A **tiny, focused model call** — ask the model one narrow question, like "does this message describe a problem needing customer support? Answer only 'triage' or 'concierge'." Use `max_tokens=5`, since you only need one word back. Strip and lowercase whatever comes back before comparing — models sometimes add a trailing period or extra words even when told not to.

Then rename your existing functions to match the file layout in the README:

```
def concierge_agent(client: OpenAI, history: list[dict], message: str) -> list[dict]:
def triage_agent(client: OpenAI, message: str) -> SupportTicket:
```

Both are thin wrappers around what you already built in Steps 2 and 3 — you're not rewriting their logic, just giving each one its own file, its own system prompt, and a name that reflects it's an independent agent now.

Log every decision — `logger.info("routed message to %s: %r", decision, message)` — not just make it. This is what makes routing checkable later, exactly like the README's Step 4 item 4 asks for. `main.py`'s loop should no longer branch on `/extract` itself. It should call `decide(message)` and let the *result* pick the branch.

Sketch `main.py`'s new loop shape — `decision = decide(message)`, then branch — before writing the router's body.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

A binary router always picks one of two agents — even when it genuinely can't tell. Look back at the README's own tricky test input: *"just wondering what your refund policy is, no rush."* That's really a Concierge-style question, but it contains "refund," a Triage trigger word. A keyword router gets it wrong outright. Even the model-call version, forced to answer only "concierge" or "triage," has to guess *something* on a message like this — it can't say "I'm not sure."

A more honest router recognizes a third outcome: **unclear**. Instead of forcing a pick, it asks the user a short clarifying question and waits for a real answer before routing. This only makes sense for the model-call version — a keyword check has no real notion of confidence, it either matched a word or it didn't.

Alongside that, log the *confidence*, not just the final decision. "routed to triage" tells you what happened. "routed to triage, 90% confident, because the message mentions being charged twice" tells you whether to trust it. That distinction matters once you're auditing routing decisions later — same idea as Project 2's trace log, mentioned in the README's Step 4 section. A router that's always "confident" even when it's guessing is a router you can't debug.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate build a router that always picks Concierge or Triage, even on a message it can't really tell about. Advanced adds a genuine third path for when the router should say "I don't know" instead of guessing, and starts logging *how sure* the router was, not just what it decided — the difference between a decision you can audit and one you can only observe.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

The plan, in plain steps:

```
function decide(message):
    look for complaint-style words in the message
    if any are found: return "triage"
    otherwise: return "concierge"
    log the decision

function concierge_agent(history, message): same as Step 2's chat, renamed
function triage_agent(message): same as Step 3's extraction, renamed

main loop:
    read what the user typed
    decision = decide(message)
    if decision is "triage":
        run triage_agent, print the ticket
    else:
        run concierge_agent, stream the reply
```

Here's almost the whole router — try finishing the rest yourself:

```
TRIGGER_WORDS = ["charged", "broken", "refund", "cancel", "not working"]

def decide(message):
    lowered = message.lower()
    if any(word in lowered for word in TRIGGER_WORDS):
        return "triage"
    return "concierge"
```

What's missing: the `Literal` type hint, the logging line, and the `main.py` loop that calls `decide()` and branches on the result.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The same plan, closer to real structure:

```
# agents/router.py
import logging
from typing import Literal

logger = logging.getLogger(__name__)

TRIGGER_WORDS = ["charged", "broken", "refund", "cancel", "not working", "can't l

def decide(message: str) -> Literal["concierge", "triage"]:
    lowered = message.lower()
    if any(word in lowered for word in TRIGGER_WORDS):
        decision: Literal["concierge", "triage"] = "triage"
    else:
        decision = "concierge"
    logger.info("routed message to %s: %r", decision, message)
    return decision
```

```
# agents/concierge_agent.py
def concierge_agent(client, history, message):
    history.append({"role": "user", "content": message})
    return stream_message(client, history)  # reuses Step 2's function

# agents/triage_agent.py
def triage_agent(client, message):
    return triage_extract(client, message)  # reuses Step 3's function
```

```
# main.py
history = []
while True:
    message = input("You: ")
    decision = decide(message)
    if decision == "triage":
        ticket = triage_agent(client, message)
        print(ticket)
```

```
else:  
    history = concierge_agent(client, history, message)
```

Notice `concierge_agent` and `triage_agent` are thin — they call the exact same underlying functions from Steps 2 and 3. The only genuinely new code in this step is the router and the loop that consults it. Test it with a mix of clear and unclear messages, including the README's ambiguous refund-policy example, before moving to Advanced.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Here's the shape of a 3-way router — try finishing the `main.py` branch for `"unclear"` yourself:

```
from typing import Literal

RouterDecision = Literal["concierge", "triage", "unclear"]

def decide_with_confidence(client: OpenAI, message: str) -> tuple[RouterDecision, str]:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {
                "role": "system",
                "content": (
                    "Decide how to route this message. Reply with exactly one line: "
                    "'concierge', 'triage', or 'unclear' followed by a short reason. "
                    "Example: 'like: triage - customer says they were charged twice. ' "
                    "Use 'unclear' only if you genuinely cannot tell which one fits."
                ),
            },
            {"role": "user", "content": message},
        ],
        max_tokens=30,
    )
    raw = response.choices[0].message.content.strip()
    decision_word, _, reason = raw.partition(" - ")
    decision_word = decision_word.strip().lower()
    if decision_word not in ("concierge", "triage", "unclear"):
        decision_word = "unclear"
    logger.info("routed message to %s (%s): %r", decision_word, reason.strip(), message)
    return decision_word, reason.strip()
```

```
# main.py
decision, reason = decide_with_confidence(client, message)
if decision == "triage":
    ticket = triage_agent(client, message)
    print("Ticket:", ticket)
elif decision == "concierge":
    history = concierge_agent(client, history, message)
```

```
else:
    # your turn: print something that asks the user to clarify,
    # instead of guessing which agent should handle it
    ...
```

Run the README's deliberately unclear test message through this version and compare what it does against the binary router from Intermediate — that comparison is the actual point of this hint.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate's `decide()` always returns exactly one of two values, no matter how uncertain the underlying signal was. Advanced adds a real third outcome, a short logged reason for every decision (not just the decision itself), and a `main.py` branch that asks rather than guesses when the router says it doesn't know.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)